

EPR Imaging

Complete Software Documentation

A bottom-up guide for Python users

This handbook explains the program from fundamental vocabulary and stored data, through services and scientific algorithms, to worker threads and the desktop interface. It covers every Python module and every MATLAB analysis script, shows representative source excerpts, and traces complete workflows from imported files to final products.

Python source	32,030 lines · 38 modules
MATLAB source	1,817 lines · 4 scripts
Primary architecture	Qt GUI → plan → worker → services → artifacts
Documentation build	2026-07-24

1. How to read this handbook

Begin with the vocabulary and data model even if your goal is to change a button. The interface is the outermost layer; the meanings of Job, Task, Scope, interval, and interpolated table are established lower down. Once those objects are clear, the larger modules become much less intimidating.

Reading source code

A function definition beginning with `def` names an operation. A class groups state and related operations. `self` means the current instance. A return value is the result sent back to the caller. A type annotation such as `list[Path]` documents the intended kind of value but usually does not perform runtime validation.

A minimal Python mental model

- Modules are `.py` files. Importing a module executes its top-level definitions and makes its names available.
- Objects carry state. Calling `object.method(...)` asks that object to perform an operation using its state.
- Dataclasses are record-like objects. In EPR Imaging they are the vocabulary shared by the GUI, persistence, planner, and processing services.
- Callbacks are functions passed into another function. Services accept log and progress callbacks so they remain independent of Qt.
- Exceptions represent failures. Worker objects catch them at the GUI boundary and emit an error signal instead of freezing the interface.

Scientific-library orientation

DataFrame

A pandas table: named columns and numbered rows, similar to a spreadsheet but designed for code and large numerical datasets.

interpolation

Estimating a value between measured samples. It is what allows navigation, sensor readings, and video frames recorded at different instants to share one timeline.

UTM

A map projection that converts longitude/latitude angles to local easting and northing distances in metres.

raster

A rectangular grid of cells. Each cell stores a value such as depth, methane concentration, or temperature.

point cloud

A collection of points in 3-D space. Every point can also carry a scalar value or RGB color.

GeoTIFF

A TIFF image whose metadata says where its pixels lie on Earth.

NetCDF

A portable scientific-data file containing named multidimensional arrays and metadata.

IDW

Inverse-distance weighting; nearby observations influence an estimated grid cell more than distant observations.

2. The problem the software solves

An underwater survey rarely produces one tidy file. Cameras record frames; navigation instruments record position and attitude; chemical and physical sensors record values; and every instrument samples at its own rate. The first job of the application is therefore temporal alignment. It converts timestamps to one numerical scale, orders and cleans every source, and estimates all requested values at a common series of times.

The central table is `interp_full.csv`. Think of it as the application's common language. A row identifies an instant; columns hold file/frame identity, navigation, projected UTM coordinates, and sensor channels. Jobs filter this table to selected intervals. Most scientific products are projections, grids, reports, or images derived from one of these tables.

Important invariants

- All naive datetime values mean UTC. They must not be interpreted in the computer's local timezone.
- Longitude/latitude are geographic coordinates; UTM easting/northing are projected metres. Algorithms that use Euclidean distance require the latter.
- A Job is a named collection of closed time ranges plus a settings snapshot. It is not itself an execution thread or output file.
- A Task states intent. A plan step is the concrete instruction produced after target and channel expansion.
- Services avoid Qt. The GUI/service seam is formed by workers and callback functions.
- Outputs are versioned into run directories when overwriting would destroy useful provenance.

```
def utc_now() -> datetime:
    """Current UTC time as a NAIVE datetime (drop-in for datetime.utcnow())."""
    return datetime.now(timezone.utc).replace(tzinfo=None)

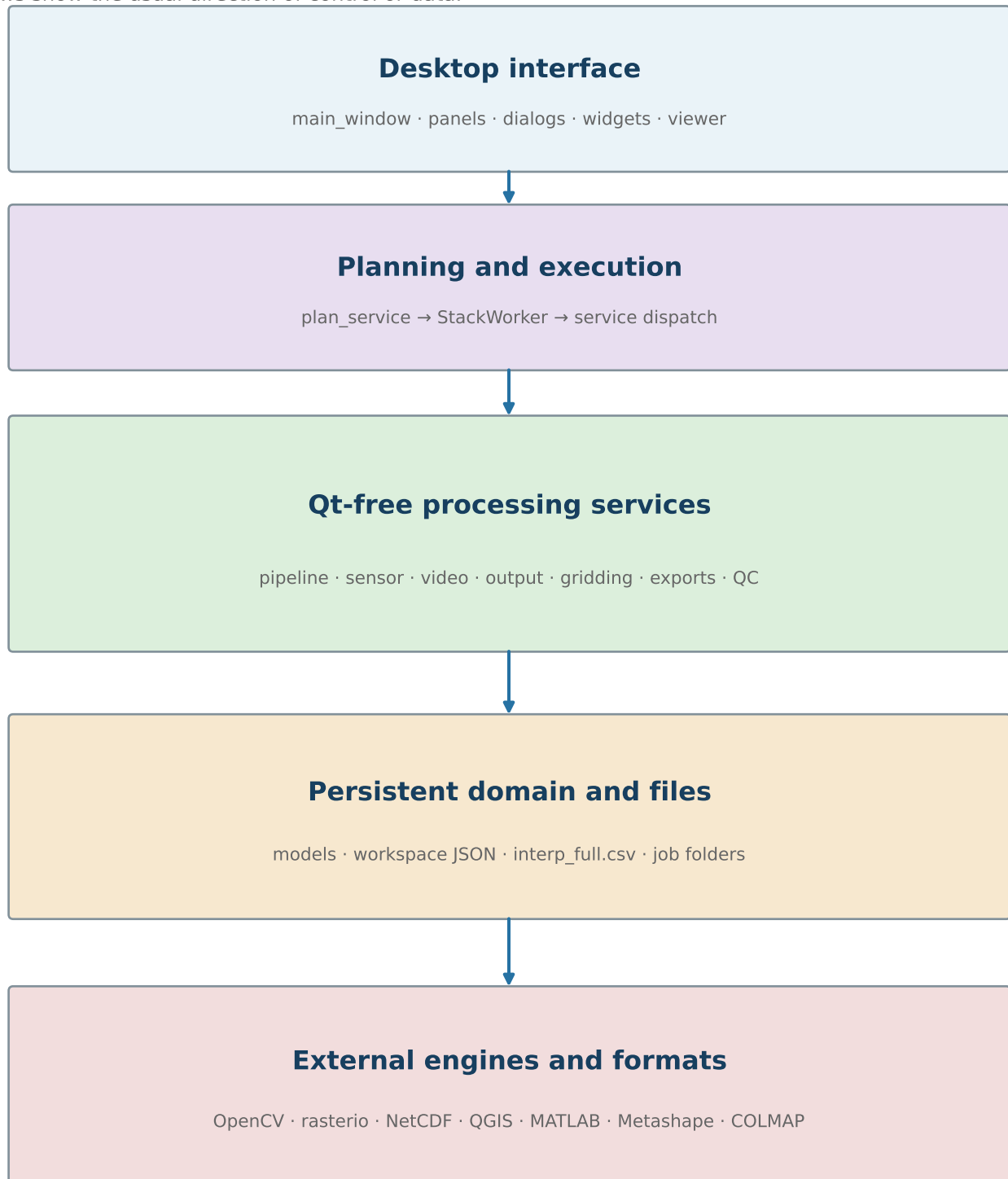
def utc_from_timestamp(timestamp: float) -> datetime:
    """Unix seconds → NAIVE UTC datetime.

    Drop-in for datetime.utcnow(). The result carries no tzinfo, which
    is what the rest of the app expects.
    """
    return datetime.fromtimestamp(float(timestamp), timezone.utc).replace(tzinfo=None)
```

The two time helpers preserve naive-UTC behavior explicitly.

System architecture at one glance

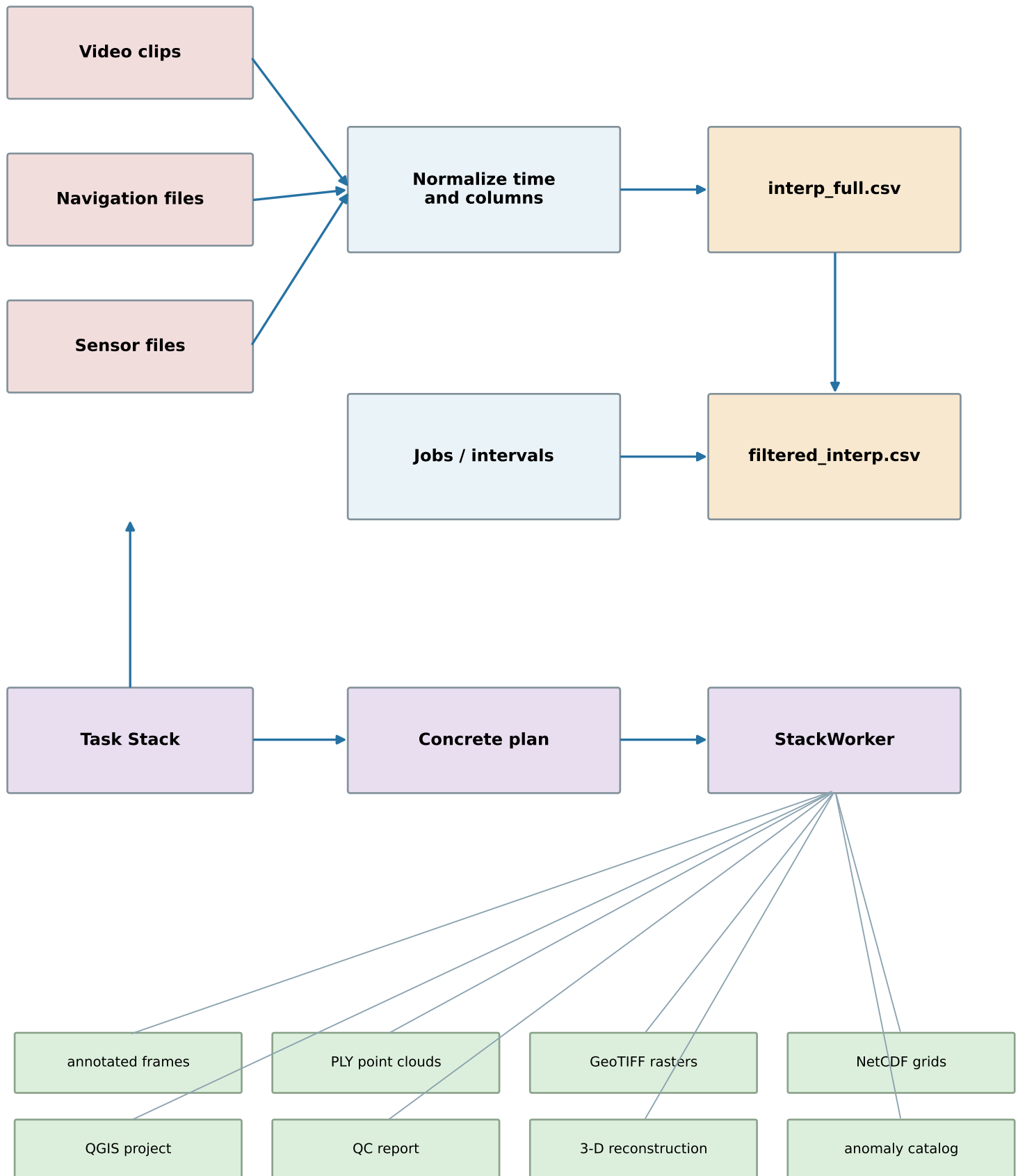
Arrows show the usual direction of control or data.



The central idea

The interface does not implement scientific algorithms itself. It collects user choices, turns them into model objects, asks the planner for concrete steps, and lets a worker invoke reusable services.

The main data pipeline



3. Workspace creation and import

At startup `app.py` constructs `MainWindow`. The user selects or creates a workspace directory. `MainWindow` delegates file inspection to `VideoService` and `SensorService`, while the two import dialogs let arbitrary source columns be mapped into the program's domain model. The workspace is saved by `ConfigService` as JSON, with internal paths stored relative to the workspace whenever possible.

- `VideoService` first attempts configured and common filename patterns, then falls back to file modification time. It reads duration and FPS from OpenCV.
- Navigation is modeled as independent time/value sources because latitude, depth, and attitude may reside in different files or use different timestamp columns.
- `SensorService` accepts ordinary delimited tables and PPI files, supports combined or separate date/time columns, and rejects unusable timestamps.
- Import dialogs preview data before committing a configuration. The configuration describes how to reload the source; it does not copy the whole table into JSON.

4. Building the master table

`PipelineService._build_full_interp_csv` is the convergence point. It creates a time axis, asks `SensorService` to load each configured source, interpolates values onto that axis, and adds UTM coordinates. The resulting CSV can be regenerated from source configuration and becomes the input to downstream task scopes.

```
def _build_full_interp_csv(
    self,
    config: "PipelineConfig",
    nav_sources: dict[str, pd.DataFrame],
    sensor_frames: list[tuple["SensorFileConfig", pd.DataFrame]],
    output_dir: Path,
) -> None:
    """Build interp_full.csv spanning the entire navigation time range.

    Unlike the per-segment interp.csv files, this output covers every
    timestamp in the navigation data at a uniform sample rate, regardless
    of job intervals. It contains no frame-extraction columns (no
    frame_filename, video_filename, or frame_index).

    The file is written to output_dir/interp_full.csv, overwriting any
    existing file.

    Args:
        config: Pipeline config (used for full_interp_sample_hz and logging).
        nav_sources: Loaded nav timeseries (lat, lon, alt, water_depth, ...).
        sensor_frames: Loaded sensor (config, dataframe) pairs.
        output_dir: Job-level output directory.
```

Representative excerpt from the live implementation. The full method also validates sources, updates progress, and writes metadata.

5. Selecting work with Jobs

Users can stage intervals manually on the map, derive them from threshold constraints, import them from CSV, or convert anomaly-catalog windows. A Job stores these intervals. `OutputService.generate_job_interval_interps` filters the master table and writes a job-local `filtered_interp.csv`, which makes each job independently inspectable and reproducible.

6. From Task to finished artifact

The Task Stack is the application's orchestration language. Task objects remain compact and editable. `plan_service` resolves each target into Scope objects, checks prerequisites, and fans per-channel tasks into multiple step dictionaries. `StackWorker` then executes those dictionaries sequentially.

```
def build_plan(ctx: PlanContext, stack: TaskStack) -> PlanResult:
    """Turn the ordered Task list into runner step dicts (preserving user order)."""
    plan: list[dict] = []
    skips: list[str] = []

    for task, sc in iter_task_scopes(ctx, stack, skips):
        scope_id, interp_path, output_dir, job = (
            sc.scope_id, sc.interp_path, sc.output_dir, sc.job
        )
        tag = f" [{sc.label}]"
        t = task.task_type
        s = task.settings
        channels = task.channels or ctx.available_channels()

        if t == "build_interp":
            config = ctx.build_interp_config(task)
            if config is None:
                continue
            plan.append({
                "label": "Build interp_full.csv",
                "product_type": "build_interp", "scope_id": "full",
                "channel": None, "method": None, "engine": None,
                "kwargs": {}, "config": config,
            })

        elif t == "sampling":
            config = ctx.build_sampling_config(task, job, scope_id)
            if config is None:
```

The planner is intentionally Qt-free, which makes target expansion testable without starting the desktop application.

- Full target: uses `interp_full.csv` and the workspace output directory.
- Job target: uses that job's filtered table and output directory.
- All-jobs target: expands once for every valid saved Job.
- Per-channel product: expands again for every selected sensor channel.
- Skip-existing mode: `StackWorker` can recognize completed versioned products and avoid recomputation.
- Failure isolation: one failed step is recorded; subsequent independent steps may continue.

Critical planning rule

Targets are resolved before execution begins. A task cannot refer to a Job that a later task will create unless that Job is materialized before the plan is built.

Threading

Long work must not run on Qt's main thread because that thread paints windows and handles input. `MainWindow` creates a `QObject` worker, moves it to `QThread`, connects typed signals to bound `QObject` methods, and starts the thread. Services receive plain callbacks from the worker and know nothing about widgets.

```
class StackWorker(QObject):
    """Executes a resolved product-stack plan in a background thread."""

    # index (1-based), total, label
    step_started = Signal(int, int, str)
    # label, list[str] of output paths
    step_finished = Signal(str, list)
    # label, error message
```

```
step_failed = Signal(str, str)
# individual log line from the underlying service
log         = Signal(str)
# per-step sub-progress 0-100 (e.g. frame extraction within a sampling step)
sub_progress = Signal(int)
# per-step detail label (e.g. "video_003.MP4 | frame 42/120")
sub_status  = Signal(str)
# summary dict: {"completed": int, "failed": int, "skipped": int, "paths": list,
```

```
#             "report": list[dict]}
finished      = Signal(dict)
# fatal error that aborts the whole stack
error         = Signal(str)

def __init__(self, plan: list[dict], log_file: Optional[str] = None,
              skip_existing: bool = False) -> None:
    super().__init__()
```

```
self._plan = plan
```

StackWorker is the deliberate bridge between Qt and the service layer.

7. Product-generation workflows

Frames

PipelineService maps desired timestamps to a source video and frame index, seeks with OpenCV, writes JPEG files, and creates a matching record. Fixed-rate sampling uses a regular clock. Dynamic sampling estimates how far the vehicle has moved and chooses spacing based on altitude and overlap targets. Optional CLAHE improves local contrast; annotations render selected metadata over the image.

Point clouds, grids, and rasters

OutputService builds a PointCloudPipeline from the filtered observations. The pipeline establishes bounds and cell size, creates a regular grid, and fills the grid from measurements. IDW is simple and local; ordinary Kriging models spatial covariance; RBF fits a smooth function. PLY retains 3-D geometry, while GeoTIFF and PNG slices expose 2-D depth bands.

NetCDF and QGIS

netcdf_export reshapes grid rows into coordinate arrays and writes dimensions, variables, fill values, and projection metadata. qgis_export discovers GeoTIFF results and writes QGIS project XML with logical groups and pseudocolor renderers. QGIS therefore consumes normal portable files; it is not embedded into the application.

Photogrammetry

photogrammetry_service is an adapter around two external reconstruction engines. Both workflows collect frames, prepare versioned output directories, use navigation to seed or align camera positions, run engine stages, and export trajectories. Metashape is driven through its Python API; COLMAP is driven through command-line programs and its SQLite database.

Anomalies

MATLAB evaluates detector strategies and exports event CSVs. Python then fuses overlapping evidence, assigns context and severity, clusters nearby windows into sites, links windows to video clips, and produces review tables, GeoJSON/QGIS assets, and a multi-page PDF catalog. Keeping CSV at the boundary means an existing detector run can be recataloged without MATLAB.

8. How the Python modules connect

The following pages cover every shipped Python module. 'Imports' names major direct dependencies visible at module scope. The inventories are extracted with Python's ast parser, so nested helper functions are not confused with public module functions.

Foundation

models.py, timeutil.py, interval_io.py, config_service.py, preset_service.py

Input and processing

video_service.py, sensor_service.py, dynamicsampling.py, pipeline_service.py

Products

point_cloud_pipeline.py, output_service.py, netcdf_export.py, qgis_export.py, qc_report.py, frame_stats.py

Orchestration

plan_service.py, stack_runner.py, anomaly_service.py, photogrammetry_service.py, build_anomaly_site_catalog.py

GUI

app.py, main_window.py, stack_panel.py, task_config_dialog.py, one_click_dialog.py, workspace_panel.py, chat_panel.py, claude_service.py, viewer_widget.py

Widgets

widgets/map_widget.py, widgets/timeline_widget.py, widgets/navigation_import_dialog.py, widgets/sensor_import_dialog.py, widgets/annotation_settings_dialog.py

Legacy example

3dvistool/point_cloud_pipeline.py, 3dvistool/example_usage.py

Documentation generator

build_architecture_pdf.py, comprehensive_software_documentation.py

3dvistool/example_usage.py

Legacy API example · 94 source lines

Responsibility

Demonstrates the standalone point-cloud pipeline in a short script.

Public shape

- Module functions: main

Connections

Internal imports: point_cloud_pipeline

Important library imports: Python standard library only.

Headless behavior

This module does not import PySide6 and can be called without constructing a QApplication.

Representative source

```
def main():
    """
    Example workflow.

    Customize the parameters below for your data.
    """

    # ===== CONFIGURATION =====
    INPUT_CSV = "your_data.csv" # Path to your input CSV file

    # Column names in your CSV
    X_COLUMN = "easting"        # UTM Easting (or X)
    Y_COLUMN = "northing"       # UTM Northing (or Y)
    Z_COLUMN = "depth"          # Depth or elevation (Z)
    SCALAR_COLUMN = "temperature" # Column to use for coloring
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

3dvistool/point_cloud_pipeline.py

Legacy standalone gridding · 515 source lines

Responsibility

An earlier self-contained point-cloud implementation retained for the separate 3dvistool example.

Public shape

- Class PointCloudPipeline: `__init__`, `load_csv`, `_calculate_bounds`, `create_3d_grid`, `to_csv`, `add_scalar_data`, `apply_cell_function`, `to_ply`, `to_raster_slices`, `summary`
- Module functions: `idw_fill`

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: `csv`, `matplotlib.cm`, `numpy`, `pandas`, `pathlib`, `plyfile`, `typing`

Headless behavior

This module does not import `PySide6` and can be called without constructing a `QApplication`.

Representative source

```
class PointCloudPipeline:
    """Pipeline for creating point cloud PLY files from 3D coordinate data."""

    def __init__(self, cell_size: float, boundary_extension: float = 0.0):
        """
        Initialize the pipeline.

        Args:
            cell_size: Size of cube cells in data units
            boundary_extension: Additional distance beyond min/max coordinates (in units)
        """
        self.cell_size = cell_size
        self.boundary_extension = boundary_extension
        self.df_grid = None
        self.bounds = None
        self.scalar_name = 'scalar'
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Gridding parameters encode scientific assumptions. Record method, cell size, neighborhood behavior, coordinate system, and scalar range beside every output.

anomaly_service.py

Anomaly workflow adapter · 430 source lines

Responsibility

Checks MATLAB availability, invokes the detector, runs the Python catalog builder, loads catalog windows, and converts selected windows into Jobs.

Public shape

- Module functions: `matlab_binary`, `matlab_unavailable_reason`, `matlab_version`, `detector_inputs_ready`, `run_detector`, `_matlab_run`, `run_catalog`, `catalog_paths`, `load_windows`, `tier_counts`, `windows_to_intervals`, `_m_escape`, `_ellipsize`, `_fmt_duration`, `_to_float`, `_to_int`

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: `csv`, `os`, `pathlib`, `shutil`, `subprocess`, `time`, `typing`

Headless behavior

This module does not import PySide6 and can be called without constructing a `QApplication`.

Representative source

```
def matlab_binary() -> Optional[str]:
    """Path to the MATLAB executable, or None if it isn't on PATH."""
    return shutil.which("matlab")

def matlab_unavailable_reason() -> Optional[str]:
    """Human-readable reason MATLAB can't be used, or None if it can.

    Mirrors photogrammetry_service.metashape_unavailable_reason() so the GUI can
    show one consistent style of explanation.
    """
    if not matlab_binary():
        return (
            "MATLAB was not found on PATH. The anomaly DETECTOR stage needs MATLAB "
            "(GrapherMatrix.m). Install MATLAB or add it to PATH; the catalog stage "
            "can still run against previously exported event CSVs."
        )
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

app.py

Application bootstrap · 69 source lines

Responsibility

Creates the Qt application, applies global appearance and icon settings, constructs MainWindow, and enters Qt's event loop.

Public shape

- Module functions: main

Connections

Internal imports: main_window

Important library imports: PySide6.QtGui, PySide6.QtWidgets, os, pathlib, sys

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
def main() -> int:
    """Create the Qt application, show the main window maximised, and run the event loop.

    Returns the integer exit code produced by QApplication.exec(), which is
    forwarded to the OS via SystemExit so that the process signals success (0)
    or failure (non-zero) correctly.
    """

    # QApplication must be constructed before any Qt widgets. sys.argv lets Qt
    # parse its own command-line flags (e.g. -platform, -style) before the rest
    # of the app sees them.
    app = QApplication(sys.argv)

    # Application name shown in window title bars, OS task-switchers, and
    # platform-specific "About" dialogs.
    app.setApplicationName("Sampling Tool")
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

build_anomaly_site_catalog.py

Anomaly catalog builder · 1,911 source lines

Responsibility

Fuses detector event CSVs, adds navigation/video context, clusters spatial sites, writes tables and GeoJSON, and lays out the anomaly review PDF.

Public shape

- Module functions: `configure`, `current_paths`, `direct_ch4_peak_events`, `load_events`, `load_navigation`, `contiguous_intervals`, `load_ctd1_flags`, `fuse_events`, `build_exact_signature_windows`, `summarize_component`, `classify_indicators`, `overlaps`, `nearest_row`, `add_context`, `haversine_m`, `assign_sites`, `iso`, `make_video_clips`, `write_qgis_layer`, `write_outputs`, `_char_widths`, `text_width_pt`, `soften`, `wrap_to_width`, `caption`, `text_page`, `table_pages`, `channel_windows`, `anomaly_mask`, `add_event_spans`...

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: `json`, `math`, `matplotlib`, `matplotlib.backends.backend_pdf`, `matplotlib.dates`, `matplotlib.pyplot`, `numpy`, `pandas`, `pathlib`, `scipy.io`, `scipy.signal`, `zipfile`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
def configure(
    repo: Path | str | None = None,
    event_root: Path | str | None = None,
    interp: Path | str | None = None,
    raw_nav: Path | str | None = None,
    ts_results: Path | str | None = None,
    out: Path | str | None = None,
) -> dict[str, Path]:
    """Point the builder at a specific workspace.

    The pipeline functions read these as module globals, so the app rebinds them
    here rather than threading a config object through every function. Any
    argument left as None keeps its current value; passing only ``repo`` re-derives
    the defaults relative to that repo.

    Returns the resolved paths, so a caller can log exactly what was used.
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

build_architecture_pdf.py

Documentation utility · 479 source lines

Responsibility

A repository utility or supporting source module.

Public shape

- Module functions: `_char_widths`, `text_width_pt`, `wrap`, `new_page`, `finish`, `heading`, `para`, `bullet`, `subhead`, `code_block`, `table`, `box`, `arrow`, `read`, `excerpt`, `module_table`, `dep_edges`, `page_title`, `page_contents`, `page_overview`

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: `ast`, `collections`, `matplotlib`, `matplotlib.backends.backend_pdf`, `matplotlib.patches`, `matplotlib.pyplot`, `pathlib`, `re`, `sys`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
def _char_widths(weight: str, family: str) -> dict:
    key = (weight, family)
    if key in _CHAR_W:
        return _CHAR_W[key]
    from matplotlib.textpath import TextPath
    from matplotlib.font_manager import FontProperties
    charset = (" !\"#$%&'()*+,-./0123456789:;<=>?@"
               "ABCDEFGHIJKLMNOPQRSTUVWXYZ[\\]^_`"
               "abcdefghijklmnopqrstuvwxyz{|}~"
               "±°→←↑↓—●■◊×…|||L.")
    prop = FontProperties(size=100.0, weight=weight, family=family)
    widths = {}
    for ch in charset:
        try:
            pair = float(TextPath((0, 0), f"n{ch}n", prop=prop).get_extents().width)
            base = float(TextPath((0, 0), "nn", prop=prop).get_extents().width)
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

chat_panel.py

Embedded assistant UI · 424 source lines

Responsibility

Renders a compact Markdown-like conversation, assembles context, streams responses, and manages API-key status.

Public shape

- Class ChatPanel: `__init__`, `_build_ui`, `eventFilter`, `refresh_status`, `_refresh_status`, `_add_welcome`, `_render_user_bubble`, `_render_assistant_bubble`, `_render_error_bubble`, `_render_view`, `_scroll_to_bottom`, `_clear_conversation`, `_send`, `_on_chunk`, `_on_finished_slot`, `_on_finished`, `_on_error`
- Module functions: `_md_to_html`

Connections

Internal imports: `claude_service`

Important library imports: `PySide6.QtCore`, `PySide6.QtGui`, `PySide6.QtWidgets`, `html`, `pathlib`, `re`, `typing`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
class ChatPanel(QWidget):
    """Streaming AI chat panel.

    Parameters:
        context_fn: callable that returns a plain-text workspace summary
                    injected into the system prompt on every send.
    """

    def __init__(self, context_fn: Callable[[], str], parent=None) -> None:
        super().__init__(parent)
        self._context_fn = context_fn

        # Conversation history in Anthropic message format
        self._history: list[dict] = []

        # Accumulates streaming text for the in-progress assistant turn
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Keep this layer focused on presentation and translation between controls and dataclasses. Expensive reads and computations should move to services or workers.

claude_service.py

Assistant API worker · 95 source lines

Responsibility

Loads and stores the Anthropic key and streams Claude messages from a QObject worker.

Public shape

- Class ClaudeWorker: `__init__`, `run`
- Module functions: `load_api_key`, `save_api_key`

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: `PySide6.QtCore`, `json`, `pathlib`, `typing`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
class ClaudeWorker(QObject):
    """Runs a single streaming Claude API call in a QThread.

    Instantiate, move to a QThread, and connect started → run.

    Signals:
        chunk(str)      - incremental text as it arrives
        finished(str)   - complete response text on clean completion
        error(str)      - error description if the call fails
    """

    chunk = Signal(str)
    finished = Signal(str)
    error = Signal(str)

    def __init__(
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

comprehensive_software_documentation.py

Documentation utility · 593 source lines

Responsibility

A repository utility or supporting source module.

Public shape

- Class Document: `__init__`, `page`, `finish`, `need`, `title`, `heading`, `wrap`, `paragraph`, `bullets`, `code`, `callout`, `close`
- Module functions: `source_files`, `module_info`, `excerpt`, `cover`, `architecture_diagram`, `dataflow_diagram`, `write_foundations`, `write_workflows`, `module_reference`, `matlab_reference`, `operations_and_glossary`, `build`

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: `ast`, `datetime`, `matplotlib`, `matplotlib.backends.backend_pdf`, `matplotlib.patches`, `matplotlib.pyplot`, `pathlib`, `re`, `textwrap`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
class Document:
    def __init__(self, pdf):
        self.pdf, self.fig, self.ax = pdf, None, None
        self.page_no = 0
        self.section = ""
        self.y = 0

    def page(self, section=None):
        if self.fig is not None:
            self.finish()
        self.fig = plt.figure(figsize=(W, H), facecolor="white")
        self.ax = self.fig.add_axes([0, 0, 1, 1])
        self.ax.set_xlim(0, W); self.ax.set_ylim(0, H); self.ax.axis("off")
        self.page_no += 1
        if section is not None:
            self.section = section
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

config_service.py

Workspace persistence · 621 source lines

Responsibility

Serializes the complete workspace, restores dataclasses, makes internal paths portable, and detects outputs from older workspaces.

Public shape

- Class ConfigService: save_json, save_workspace, load_workspace, _load_time_value_source, _load_sensor_file, _parse_dt, _load_interval, _detect_applied_steps
- Module functions: _to_relative, _resolve

Connections

Internal imports: models, timeutil

Important library imports: datetime, json, pathlib, typing

Headless behavior

This module does not import PySide6 and can be called without constructing a QApplication.

Representative source

```
class ConfigService:
    """Static-method collection for workspace file I/O.

    All methods are @staticmethod because there is no instance state; the class
    acts as a namespace. Callers use ConfigService.save_workspace() and
    ConfigService.load_workspace() directly.
    """

    # -----
    # Legacy JSON save (alongside pipeline output)
    # -----

    @staticmethod
    def save_json(
        output_path: str | Path,
        video_directory: str | Path,
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

dynamicsampling.py

Adaptive frame scheduling · 433 source lines

Responsibility

Chooses extraction times from vehicle motion and altitude so overlapping seafloor imagery is neither too sparse nor unnecessarily dense.

Public shape

- Module functions: `_series_to_epoch_seconds`, `_haversine_m`, `create_dynamic_sample_schedule`

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: `math`, `numpy`, `pandas`

Headless behavior

This module does not import `PySide6` and can be called without constructing a `QApplication`.

Representative source

```
def _series_to_epoch_seconds(s: pd.Series) -> np.ndarray:
    """Convert a pandas Series to a float64 array of Unix epoch seconds.

    Accepts two forms:
    - Numeric (int or float) columns already in Unix seconds.
    - Datetime-like strings or pandas Timestamp objects, which are converted
      via pd.to_datetime with UTC interpretation.

    Args:
        s: A pandas Series containing timestamps in either numeric or
          datetime-like format.

    Returns:
        float64 numpy array of seconds since 1970-01-01T00:00:00 UTC.
    """
    if pd.api.types.is_numeric_dtype(s):
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

frame_stats.py

Image-set quality · 171 source lines

Responsibility

Measures sharpness, brightness, contrast, and clipping for extracted JPEGs and emits CSV/JSON summaries plus plots.

Public shape

- Module functions: `image_quality`, `_safe`, `analyze_frame_set`

Connections

Internal imports: `qc_report`

Important library imports: `json`, `numpy`, `pathlib`, `re`, `typing`

Headless behavior

This module does not import `PySide6` and can be called without constructing a `QApplication`.

Representative source

```
def image_quality(path: str) -> dict:
    """Compute image-quality metrics for one frame.  {} if unreadable."""
    import cv2
    img = cv2.imread(path)
    if img is None:
        return {}
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    total = gray.size or 1
    b, g, r = (float(img[:, :, i].mean()) for i in range(3))
    return {
        "sharpness": float(cv2.Laplacian(gray, cv2.CV_64F).var()),
        "brightness": float(gray.mean()),
        "contrast": float(gray.std()),
        "pct_clipped_high": float((gray >= 250).sum()) / total * 100.0,
        "pct_clipped_low": float((gray <= 5).sum()) / total * 100.0,
        "mean_r": r, "mean_g": g, "mean_b": b,
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

interval_io.py

Interval interchange · 171 source lines

Responsibility

Imports time windows from CSV and checks whether those windows fall inside available video and data coverage.

Public shape

- Module functions: `_parse_timestamp`, `_find_column`, `parse_intervals_csv`, `coverage_warnings`

Connections

Internal imports: `models`

Important library imports: `csv`, `datetime`, `pathlib`

Headless behavior

This module does not import `PySide6` and can be called without constructing a `QApplication`.

Representative source

```
def _parse_timestamp(raw: str) -> datetime | None:
    """Parse one timestamp cell: ISO-8601 (with or without date) or unix seconds.

    Returns a naive datetime, or None when the cell can't be parsed.
    """
    text = (raw or "").strip()
    if not text:
        return None
    # Unix seconds (int or float).
    try:
        unix = float(text)
        # Guard against "20260115" style compact dates being read as epoch.
        if 1e8 <= unix <= 4e9:
            return datetime.fromtimestamp(unix, tz=timezone.utc).replace(tzinfo=None)
    except ValueError:
        pass
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

main_window.py

Desktop application coordinator · 8,845 source lines

Responsibility

Builds the principal user interface and coordinates every service, widget, worker thread, workspace state transition, and user workflow.

Public shape

- Class WorkspaceStartupDialog: `__init__`, `_choose`, `keyPressEvent`, `closeEvent`
- Class PipelineWorker: `__init__`, `run`
- Class OutputWorker: `__init__`, `run`
- Class AnomalyWorker: `__init__`, `cancel`, `run`
- Class PhotogrammetryWorker: `__init__`, `run`
- Class MainWindow: `_downsample_track`, `__init__`, `_build_ui`, `_build_toolbar`, `_build_status_bar`, `_reset_progress`, `_build_inputs_tab`, `_build_threshold_tab`, `_build_manual_interval_tab`, `_build_anomaly_tab`, `_anomaly_out_dir`, `_anomaly_interp_path`, `_anomaly_run`, `_anomaly_set_status`, `_anomaly_cancel`, `_anomaly_teardown`, `_anomaly_finished`, `_anomaly_error`, `_anomaly_reload`, `_anomaly_refresh_table` ...
- Module functions: `_app_version`, `_iso_to_unix`

Connections

Internal imports: `chat_panel`, `claude_service`, `config_service`, `models`, `pipeline_service`, `sensor_service`, `timeutil`, `video_service`, `widgets.annotation_settings_dialog`, `widgets.map_widget`, `widgets.navigation_import_dialog`, `widgets.sensor_import_dialog`, `widgets.timeline_widget`, `workspace_panel`

Important library imports: `PySide6.QtCore`, `PySide6.QtGui`, `PySide6.QtWidgets`, `calendar`, `copy`, `datetime`, `math`, `numpy`, `pandas`, `pathlib`, `pyqtgraph`, `shutil`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
class MainWindow(QMainWindow):
    """Top-level application window for the EPR Video + Sensor Processing Tool.

    Layout (left → right):
    1. Left panel: QWidget with four tabs
       - Inputs:      Video dir, navigation, sensors, time intervals
       - Processing:   Output dir, sampling mode, frame rate, options
       - Post-Processing: Per-step checkboxes, CLAHE params, thresholds
       - Visualization: CSV load, box-select, interval pick, segments
    2. Centre: QStackedWidget cycling between
       - Timeline panel (visible when Inputs / Processing tab active)
       - Graphs panel   (visible when Post-Processing tab active)
       - Map panel       (visible when Visualization tab active)
    3. Right panel: Project Summary, Skipped/Warnings, Processing Log

    The three panels sit inside a QSplitter so the user can resize them.
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

This is the largest and most coupled module. New scientific logic should normally enter a service first; MainWindow should coordinate it. Worker signal connections deserve special review because a lambda can execute in the emitting thread.

models.py

Domain model · 804 source lines

Responsibility

Defines the small dataclasses that give names and structure to videos, sensor sources, navigation, time intervals, jobs, tasks, and processing history.

Public shape

- Class VideoRecord: to_dict
- Class SensorChannel: to_dict
- Class SensorFileConfig: to_dict
- Class TimeValueSourceConfig: to_dict
- Class NavigationConfig: _all_sources, start_time, end_time, to_dict
- Class AnnotationConfig: to_dict, from_dict
- Class SelectedTimeRange: to_dict
- Class ThresholdConstraint: to_dict, from_dict
- Class ThresholdConfig: label, to_dict, from_dict
- Class Job: to_dict
- Class SegmentRecord: to_dict
- Class PhotogrammetryRun: to_dict, from_dict
- Class Task: type_label, per_channel, requires, target_label, display_label, to_dict, from_dict
- Class TaskStack: new_id, add, remove, move, get, to_dict, from_dict

Connections

Internal imports: timeutil

Important library imports: dataclasses, datetime, pathlib, typing

Headless behavior

This module does not import PySide6 and can be called without constructing a QApplication.

Representative source

```
@dataclass
class VideoRecord:
    """Represents a single video file that has been scanned from disk.

    VideoService.scan_directory() produces these objects. The pipeline uses
    them to know which files to open, when each file starts/ends in absolute
    time, and how fast to seek through frames.

    All datetime values are naive (no timezone). The convention throughout
    the application is to treat every naive datetime as UTC.
    """

    # Absolute path to the video file on disk.
    path: Path

    # Just the filename portion (e.g. "2026_01_18T15_36_28.MP4"). Kept
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the

excerpt illustrates structure, not the entire contract.

Maintenance notes

Changing a field affects JSON compatibility, dialogs, planner behavior, and workspace reload. Supply defaults and update both serialization directions.

netcdf_export.py

NetCDF writer · 192 source lines

Responsibility

Converts a gridded point table into coordinate arrays and writes a CF-oriented NetCDF file with projection metadata.

Public shape

- Module functions: `_require_netcdf4`, `_utm_grid_mapping_name`, `write_gridded_netcdf`, `grid_to_arrays`

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: `datetime`, `numpy`, `typing`

Headless behavior

This module does not import `PySide6` and can be called without constructing a `QApplication`.

Representative source

```
def _require_netcdf4():
    try:
        import netCDF4 # noqa: F401
        return netCDF4
    except ImportError as exc:
        raise ImportError(
            "NetCDF export needs the 'netCDF4' package. Install it with:\n"
            "    pip install netCDF4"
        ) from exc

def _utm_grid_mapping_name(eps_g: Optional[int]) -> str:
    # UTM (and most projected EPSG codes used here) is Transverse Mercator.
    return "transverse_mercator"
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

one_click_dialog.py

Pipeline recipe generator · 587 source lines

Responsibility

Collects a high-level set of desired products and converts it into ordinary editable Task objects for the normal execution engine.

Public shape

- Class OneClickPipelineDialog: `__init__`, `_build_ui`, `_photo_summary_text`, `_configure_photogrammetry`, `accept`, `_checked_jobs`, `_target`, `wants_anomaly_target`, `anomaly_tiers`, `wants_anomaly_step`, `_selected_channels`, `replace_stack`, `build_tasks`

Connections

Internal imports: `models`, `task_config_dialog`

Important library imports: `PySide6.QtCore`, `PySide6.QtWidgets`, `typing`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
class OneClickPipelineDialog(QDialog):
    """Assemble target + products into a ready-to-run task stack."""

    def __init__(
        self,
        jobs: list[tuple],          # [(job_id, name), ...] with intervals
        channels: list[str],        # sensor channels from interp_full.csv
        availability: dict,         # from MainWindow._stack_data_availability
        interp_exists: bool,
        anomaly_info: Optional[dict] = None, # {"tier_counts", "matlab_reason", "catalog_exists"}
        parent: Optional[QWidget] = None,
    ) -> None:
        super().__init__(parent)
        self._jobs = jobs
        self._channels = channels
        self._avail = availability or {}
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Keep this layer focused on presentation and translation between controls and dataclasses. Expensive reads and computations should move to services or workers.

output_service.py

Science-product façade · 1,192 source lines

Responsibility

Presents one consistent API for point clouds, rasters, slices, NetCDF, QGIS, QC reports, and per-job filtered tables.

Public shape

- Class OutputService: `__init__`, `_next_run_dir`, `_save_grid_csv`, `_pipeline_from_grid_csv`, `_save_meta`, `_build_nav_grid`, `_build_sensor_grid`, `generate_nav_3d_ply`, `generate_sensor_3d_ply`, `generate_sensor_netcdf`, `generate_nav_slices_from_run`, `generate_sensor_slices_from_run`, `generate_qgis_project`, `generate_nav_2d_geotiff`, `generate_sensor_2d_geotiff`, `generate_depth_slice_geotiffs`, `generate_qc_report`, `_utm_epsg_from_df`, `_write_geotiff`, `generate_job_interval_interps`
- Module functions: `sensor_channels_from_csv`, `_utm_zone_to_epsg`, `_check_utm_cols`, `_neg_depth_csv`

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: `contextlib`, `datetime`, `json`, `numpy`, `os`, `pandas`, `pathlib`, `tempfile`, `typing`

Headless behavior

This module does not import `PySide6` and can be called without constructing a `QApplication`.

Representative source

```
class OutputService:
    """Generates dataset-level outputs (GeoTIFF, PLY) from interp_full.csv."""

    def __init__(self, log_fn=None):
        self._log = log_fn or print

    # -----
    # Run-directory helpers
    # -----

    @staticmethod
    def _next_run_dir(parent: Path) -> Path:
        """Create and return the next numbered run_NNN subdirectory under parent.

        Each call to a generate method creates a new versioned directory so
        previous outputs are never overwritten. Run numbers start at 001 and
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Treat paths, columns, units, nodata values, and progress callbacks as part of the public contract. These services are used both from direct GUI actions and the Task Stack.

photogrammetry_service.py

3-D reconstruction adapters · 1,467 source lines

Responsibility

Detects Metashape and COLMAP, prepares versioned runs, processes images, seeds camera positions, and exports trajectories and models.

Public shape

- Module functions: `_metashape_major`, `_meta_api`, `_is_wsl`, `detect_engines`, `_detect_metashape`, `metashape_unavailable_reason`, `_find_windows_metashape_exe`, `_find_metashape_exe`, `_detect_colmap`, `prepare_run_dir`, `save_meta`, `_fsize`, `_stage_timer`, `_progress_logger`, `_call_with_progress`, `build_chunk_sets`, `run_metashape`, `_process_metashape_chunk`, `run_metashape_batch`, `_collect_frames`, `_seed_camera_locations`, `_timestamp_from_camera_label`, `_export_cameras_json`, `launch_in_metashape`, `run_colmap`, `_build_colmap_geo_txt`, `_inject_db_positions`, `_export_colmap_trajectory`, `_camera_center`, `_colmap_run` ...

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: `contextlib`, `datetime`, `json`, `os`, `pathlib`, `platform`, `shutil`, `subprocess`, `sys`, `time`, `typing`

Headless behavior

This module does not import PySide6 and can be called without constructing a `QApplication`.

Representative source

```
def _metashape_major(Metashape) -> int:
    """Return the Metashape major version (e.g. 2), defaulting to 2 if unknown."""
    for getter in (lambda: Metashape.app.version, lambda: Metashape.version):
        try:
            return int(str(getter()).split(".")[0])
        except Exception:
            continue
    return 2 # assume modern API when the version can't be read

def _meta_api(Metashape) -> dict:
    """Resolve version-appropriate dense/sparse bindings.

    Returns a dict with:
        build_dense    - callable(chunk) that builds the dense/point cloud
        dense_source   - DataSource enum for the dense cloud
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

pipeline_service.py

Core survey pipeline · 2,432 source lines

Responsibility

Builds the master interpolated table, finds threshold intervals, extracts frames, annotates images, applies CLAHE, and creates legacy raster products.

Public shape

- Class PipelineConfig: record fields only
- Class PipelineService: `__init__`, `log`, `run`, `find_threshold_intervals`, `_intervals_overlap`, `_file_dt_to_unix`, `_extract_frames_for_interval`, `_create_sampled_frame_df`, `_build_nav_df`, `_get_dynamic_sample_times`, `_create_dynamic_sample_df`, `_apply_thresholds`, `_split_contiguous_groups`, `_extract_frames_for_timestamps`, `_build_full_interp_csv`, `_build_master_dataframe`, `_update_master_dataframe`, `_add_utm_columns`, `_sensor_value_columns`, `_create_sensor_raster` ...

Connections

Internal imports: `dynamicsampling`, `models`, `sensor_service`, `timeutil`, `video_service`

Important library imports: `calendar`, `cv2`, `dataclasses`, `datetime`, `logging`, `math`, `numpy`, `pandas`, `pathlib`, `rasterio`, `rasterio.transform`, `scipy.interpolate`, `typing`

Headless behavior

This module does not import `PySide6` and can be called without constructing a `QApplication`.

Representative source

```
class PipelineService:
    """Stateless processing service that drives every pipeline stage.

    The service itself holds almost no state – only a logger and an optional
    log function injected at construction time. All per-run state travels
    in the PipelineConfig argument. This design makes PipelineService safe
    to reuse across multiple run() calls from the same GUI session.

    Typical call sequence (from a background QThread):
        service = PipelineService(log_fn=self.log_signal.emit)
        produced = service.run(config)
        # produced is a list of Path objects for the directories that were written
    """

    def __init__(self, log_fn: LogFn | None = None):
        """Initialise with an optional external log sink.
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Treat paths, columns, units, nodata values, and progress callbacks as part of the public contract. These services are used both from direct GUI actions and the Task Stack.

plan_service.py

Task planner · 473 source lines

Responsibility

Resolves abstract task targets into concrete full-survey or per-job scopes, validates requirements, and expands tasks into executable steps.

Public shape

- Class Scope: record fields only
- Class PlanContext: record fields only
- Class PlanResult: `__len__`, `__iter__`, `__bool__`
- Module functions: `available_jobs`, `find_job`, `scope_full`, `scope_for_job`, `resolve_task_scopes`, `iter_task_scopes`, `step`, `build_plan`

Connections

Internal imports: models

Important library imports: calendar, dataclasses, pathlib, typing

Headless behavior

This module does not import PySide6 and can be called without constructing a QApplication.

Representative source

```
class Scope:
    """One concrete execution context for a task."""
    scope_id: str          # "full" | "job_<id>"
    interp_path: str       # sensor table this scope reads
    output_dir: str        # where products are written
    label: str             # human-readable, used in step labels
    job: Optional[Job]     # None for the full-dataset scope

@dataclass
class PlanContext:
    """Everything the planner needs from the application.

    Data fields are read directly; the callables cover work that depends on GUI
    state (or on disk) and is therefore owned by MainWindow.
    """
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Planning should remain deterministic and side-effect free. A plan can be inspected, counted, and tested before any expensive task runs.

point_cloud_pipeline.py

Grid and point-cloud engine · 846 source lines

Responsibility

Projects observations into a regular 3-D grid, fills missing values with IDW, Kriging, or RBF, and writes CSV, PLY, and raster slices.

Public shape

- Class PointCloudPipeline: `__init__`, `_logln`, `load_csv`, `_calculate_bounds`, `create_3d_grid`, `to_csv`, `add_scalar_data`, `apply_cell_function`, `to_ply`, `to_raster_slices`, `summary`
- Module functions: `resolve_cmap`, `write_colorbar_legend`, `idw_fill`, `ordinary_kriging_fill`, `rbf_fill`

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: PIL, csv, matplotlib, numpy, pandas, pathlib, plyfile, typing

Headless behavior

This module does not import PySide6 and can be called without constructing a QApplication.

Representative source

```
class PointCloudPipeline:
    """Pipeline for creating point cloud PLY files from 3D coordinate data."""

    def __init__(self, cell_size: float, boundary_extension: float = 0.0,
                 log_fn: Optional[Callable[[str], None]] = None):
        """
        Initialize the pipeline.

        Args:
            cell_size: Size of cube cells in data units
            boundary_extension: Additional distance beyond min/max coordinates (in units)
            log_fn: Optional callable(str) for progress messages. Defaults to
                    self._logln(); pass the app's log callback to route output to the GUI.
        """
        self.cell_size = cell_size
        self.boundary_extension = boundary_extension
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Gridding parameters encode scientific assumptions. Record method, cell size, neighborhood behavior, coordinate system, and scalar range beside every output.

preset_service.py

Reusable task templates · 138 source lines

Responsibility

Stores and restores user-named task and stack templates in JSON without coupling them to a particular task identifier.

Public shape

- Module functions: `presets_path`, `load_presets`, `save_presets`, `task_to_template`, `template_to_task`, `list_task_templates`, `add_task_template`, `delete_task_template`, `list_stack_templates`, `add_stack_template`, `delete_stack_template`

Connections

Internal imports: `models`

Important library imports: `json`, `pathlib`, `typing`

Headless behavior

This module does not import `PySide6` and can be called without constructing a `QApplication`.

Representative source

```
def presets_path() -> Path:
    """Path to presets.json in the user config dir (mirrors last_session.json)."""
    cfg_dir = Path.home() / ".config" / "epr_imaging"
    cfg_dir.mkdir(parents=True, exist_ok=True)
    return cfg_dir / "presets.json"

def load_presets() -> dict[str, list]:
    """Load all presets; returns {'task_templates': [...], 'stack_templates': [...]}."""
    path = presets_path()
    if not path.exists():
        return {"task_templates": [], "stack_templates": []}
    try:
        data = json.loads(path.read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError):
        return {"task_templates": [], "stack_templates": []}
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

qc_report.py

Data-quality reporting · 313 source lines

Responsibility

Calculates per-channel statistics, gaps, temporal coverage, histograms, and a machine-readable quality summary.

Public shape

- Module functions: `channel_stats`, `detect_gaps`, `coverage_analysis`, `write_histogram_png`, `_fmt_dt`, `generate_report`

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: PIL, json, numpy, pandas, pathlib, typing

Headless behavior

This module does not import PySide6 and can be called without constructing a `QApplication`.

Representative source

```
def channel_stats(values: np.ndarray) -> dict:
    """Return summary statistics for one channel's per-frame values.

    NaN entries are treated as missing and excluded from the numeric stats but
    counted in n_missing.
    """
    values = np.asarray(values, dtype=float)
    n_total = int(values.size)
    finite = values[np.isfinite(values)]
    n_valid = int(finite.size)
    n_missing = n_total - n_valid

    if n_valid == 0:
        return {
            "n_total": n_total, "n_valid": 0, "n_missing": n_missing,
            "min": None, "max": None, "mean": None, "median": None, "std": None,
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

qgis_export.py

QGIS project writer · 394 source lines

Responsibility

Discovers produced GeoTIFFs and writes QGIS XML containing grouped raster layers and pseudocolor styling.

Public shape

- Module functions: `generate_qgis_project`, `_collect_tifs`, `_collect_sensor_2d`, `_collect_sensor_slices`, `_depth_label`, `_read_epsg`, `_raster_min_max`, `_new_id`, `_srs_el`, `_add_project_crs`, `_tree_group`, `_add_tree_layer`, `_make_raster_layer`, `_add_pseudocolor_pipe`, `_indent_xml`

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: `numpy`, `pathlib`, `re`, `typing`, `uuid`, `xml.etree.ElementTree`

Headless behavior

This module does not import `PySide6` and can be called without constructing a `QApplication`.

Representative source

```
def generate_qgis_project(
    output_dir: str,
    project_name: str = "EPR Survey",
) -> str:
    """Scan output_dir for GeoTIFFs and write a .qgs project file.

    Returns the absolute path of the written .qgs file.

    Raises FileNotFoundError if no GeoTIFF files are found.
    Raises RuntimeError if rasterio is not available (needed for min/max/CRS).
    """
    try:
        import rasterio
    except ImportError:
        raise RuntimeError("rasterio is required for QGIS project generation.")
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

sensor_service.py

Tabular input normalization · 716 source lines

Responsibility

Reads CSV/PPI sources, repairs common coordinate formats, normalizes timestamps, and interpolates sensor and navigation values.

Public shape

- Class SensorService: `_is_ppi`, `_read_ppi`, `_read_file`, `read_preview`, `read_columns`, `_fix_decimal_minutes`, `normalize_timestamps`, `_parse_separate_date_time`, `_get_timestamp_series`, `build_config`, `build_time_value_source_config`, `build_navigation_config`, `load_sensor_dataframe`, `load_time_value_dataframe`, `build_sensor_raster_dataframe`, `interpolate_series`

Connections

Internal imports: `models`

Important library imports: `numpy`, `pandas`, `pathlib`, `re`

Headless behavior

This module does not import `PySide6` and can be called without constructing a `QApplication`.

Representative source

```
class SensorService:
    """Static-method collection for loading and processing sensor/nav CSV data.

    All methods are @staticmethod because there is no instance state; the class
    acts as a namespace. Callers import SensorService and call methods directly
    without instantiating it.
    """

    # Common column names that instruments use for their timestamp column.
    # Used by the import dialog to auto-select the timestamp column when
    # presenting a preview of a new file.
    TIMESTAMP_CANDIDATES = [
        "timestamp",
        "time",
        "datetime",
        "date_time",
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

stack_panel.py

Task-stack editor · 488 source lines

Responsibility

Lets users add, reorder, duplicate, edit, remove, save, and run tasks while showing requirement and failure state.

Public shape

- Class StackPanel: `__init__`, `set_providers`, `set_running`, `skip_existing`, `set_failed_count`, `_build_ui`, `refresh`, `_task_tooltip`, `_selected_task_id`, `_show_create_menu`, `_sampling_tasks_for_dialog`, `_create_task`, `_edit_selected`, `_remove_selected`, `_move_selected`, `_duplicate_selected`, `_show_templates_menu`, `_save_task_template`, `_save_stack_template`, `_insert_task_from_template` ...
- Class ManageTemplatesDialog: `__init__`, `_reload`, `_delete_task`, `_delete_stack`

Connections

Internal imports: `models`, `preset_service`, `task_config_dialog`

Important library imports: `PySide6.QtCore`, `PySide6.QtGui`, `PySide6.QtWidgets`, `typing`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
class StackPanel(QWidget):
    """Dockable task-stack editor + runner."""

    run_requested      = Signal()
    rerun_failed_requested = Signal()
    tasks_changed      = Signal() # emitted whenever the stack is mutated
    one_click_requested = Signal() # open the One-Click Pipeline builder

    def __init__(self, stack: TaskStack, parent: Optional[QWidget] = None) -> None:
        super().__init__(parent)
        self._stack = stack
        # Provider callables, injected via set_providers().
        self._jobs_provider: Callable[[], list[tuple]] = lambda: []
        self._channels_provider: Callable[[], list[str]] = lambda: []
        self._availability_provider: Callable[[], dict] = lambda: {}
        self._build_ui()
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Keep this layer focused on presentation and translation between controls and dataclasses. Expensive reads and computations should move to services or workers.

stack_runner.py

Sequential executor · 880 source lines

Responsibility

Runs a prebuilt plan in a worker thread, dispatches each product type to the correct service, isolates failures, logs outputs, and supports cancellation.

Public shape

- Class StackWorker: `__init__`, `abort`, `_ts`, `_file_write`, `_emit`, `_sampling_status`, `_sampling_substatus`, `_logln`, `_log_event`, `run`, `_log_environment_header`, `_record_skipped_sampling_outputs`, `_log_step_header`, `_log_outputs`, `_already_done`, `_run_step`, `_run_slices`, `_grid_scalar_range`, `_run_build_interp`, `_run_anomaly_detect`...

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: `PySide6.QtCore`, `pathlib`, `time`, `typing`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
class StackWorker(QObject):
    """Executes a resolved product-stack plan in a background thread."""

    # index (1-based), total, label
    step_started = Signal(int, int, str)
    # label, list[str] of output paths
    step_finished = Signal(str, list)
    # label, error message
    step_failed = Signal(str, str)
    # individual log line from the underlying service
    log = Signal(str)
    # per-step sub-progress 0–100 (e.g. frame extraction within a sampling step)
    sub_progress = Signal(int)
    # per-step detail label (e.g. "video_003.MP4 | frame 42/120")
    sub_status = Signal(str)
    # summary dict: {"completed": int, "failed": int, "skipped": int, "paths": list,
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Dispatch names must stay aligned with `TASK_INFO` and plan step `product_type` values. Ensure every external resource and log file is finalized on success, error, and cancellation.

task_config_dialog.py

Task settings editor · 994 source lines

Responsibility

Builds type-specific forms for task targets, channels, gridding, sampling, slices, anomaly detection, and photogrammetry.

Public shape

- Class TaskConfigDialog: `__init__`, `_restore_target_selection`, `_collect_target`, `_dspin`, `_combo`, `_check`, `_build_settings_form`, `_build_photogrammetry_form`, `_browse_frame_dir`, `accept`, `task`

Connections

Internal imports: `models`

Important library imports: `PySide6.QtCore`, `PySide6.QtWidgets`, `typing`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
class TaskConfigDialog(QDialog):
    """Configure one Task's target, channels, and settings."""

    def __init__(
        self,
        task: Task,
        available_jobs: list[tuple],          # [(job_id, name), ...]
        available_channels: list[str],
        stack_sampling_tasks: list[tuple] = (), # [(task_id, display_label), ...]
        parent: Optional[QWidget] = None,
    ) -> None:
        super().__init__(parent)
        self._task = task
        self._jobs = available_jobs
        self._channels = available_channels
        self._sampling_tasks = list(stack_sampling_tasks)
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Keep this layer focused on presentation and translation between controls and dataclasses. Expensive reads and computations should move to services or workers.

timeutil.py

UTC compatibility helpers · 49 source lines

Responsibility

Centralizes creation of naive UTC datetimes so old and new Python versions preserve the application's time convention.

Public shape

- Module functions: `utc_now`, `utc_from_timestamp`

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: `datetime`

Headless behavior

This module does not import `PySide6` and can be called without constructing a `QApplication`.

Representative source

```
def utc_now() -> datetime:
    """Current UTC time as a NAIVE datetime (drop-in for datetime.utcnow())."""
    return datetime.now(timezone.utc).replace(tzinfo=None)

def utc_from_timestamp(timestamp: float) -> datetime:
    """Unix seconds → NAIVE UTC datetime.

    Drop-in for datetime.utcfromtimestamp(). The result carries no tzinfo, which
    is what the rest of the app expects.
    """
    return datetime.fromtimestamp(float(timestamp), timezone.utc).replace(tzinfo=None)
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

video_service.py

Video discovery · 315 source lines

Responsibility

Scans a directory, recognizes supported clips, infers each clip's start time, and reads duration and frame-rate metadata with OpenCV.

Public shape

- Class VideoScanError: record fields only
- Class VideoService: `__init__`, `scan_directory`, `_parse_start_time`, `_read_video_duration`, `_build_regex_from_format`

Connections

Internal imports: `models`

Important library imports: `cv2`, `datetime`, `pathlib`, `re`

Headless behavior

This module does not import `PySide6` and can be called without constructing a `QApplication`.

Representative source

```
class VideoScanError(Exception):
    """Raised when a video directory is missing or a file cannot be opened.

    Callers catch this to report problems in the UI rather than crashing.
    """
    pass

class VideoService:
    """Scans a directory of video files, parses start times, and reads metadata.

    One instance is created per scan (because the datetime_format can change
    between scans). The two main entry points are:
    - scan_directory() - returns (list[VideoRecord], list[str skipped])
    - _parse_start_time() - internal, but documented below for reference
    """
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

viewer_widget.py

Interactive 3-D viewer · 869 source lines

Responsibility

Loads point clouds and meshes into PyVista, controls layer visibility/color/opacity and level of detail, captures images, and exports Potree scenes.

Public shape

- Class PotreeWorker: `__init__`, `run`, `_find_html`
- Class Layer: `__init__`, `display_name`, `count_label`
- Class LayerRowWidget: `__init__`
- Class PointCloudViewer: `__init__`, `_build_ui`, `load_file`, `reload_layer_full`, `_on_budget_changed`, `remove_layer`, `_mesh_has_rgb`, `_maybe_downsample`, `_render_layer`, `_promote_to_lod`, `_point_count`, `_refresh_row_for`, `_add_list_row`, `_on_visibility`, `_on_opacity`, `_on_color_mode`, `_on_log_scale`, `_add_layer_dialog`, `_reset_camera`, `_toggle_background` ...
- Module functions: `detect_potree_converter`, `get_viewer`, `viewer_available`

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: `PySide6.QtCore`, `PySide6.QtGui`, `PySide6.QtWidgets`, `os`, `pathlib`, `typing`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
class PotreeWorker(QObject):
    """Runs PotreeConverter on a point cloud in a background thread.

    PotreeConverter 1.x can emit a self-contained HTML page (--generate-page);
    2.x produces an octree that needs a Potree page to view. We try the
    1.x page-generating form first and fall back to a bare conversion,
    reporting whichever output we can open.
    """

    finished = Signal(str) # path to open (html file or output dir)
    error = Signal(str)
    log = Signal(str)

    def __init__(self, converter: str, input_path: str, output_dir: str):
        super().__init__()
        self.converter = converter
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Preserve the module's current boundary. When adding an option, trace it from model/configuration through planning and execution to output metadata.

widgets/annotation_settings_dialog.py

Frame-overlay editor · 302 source lines

Responsibility

Configures which navigation/sensor values appear on extracted images and controls font, placement, colors, and background.

Public shape

- Class AnnotationSettingsDialog: `__init__`, `_build_available_items`, `_build_ui`, `_on_bg_toggled`, `_hex_color`, `_update_color_button`, `_pick_color`, `get_result`

Connections

Internal imports: `models`

Important library imports: `PySide6.QtCore`, `PySide6.QtGui`, `PySide6.QtWidgets`, `typing`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
class AnnotationSettingsDialog(QDialog):
    """Dialog for configuring frame annotation content and appearance.

    Shows all available annotation fields based on the current navigation
    and sensor configuration. Built-in fields (filename, timestamp, nav
    channels) are always listed; sensor channel fields are added from the
    configured sensor files. The user can check any combination.

    Usage:
        dlg = AnnotationSettingsDialog(
            parent,
            current_config=self.annotation_config,
            navigation_file=self.navigation_file,
            sensor_files=self.sensor_files,
        )
        if dlg.exec() == QDialog.Accepted:
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Keep this layer focused on presentation and translation between controls and dataclasses. Expensive reads and computations should move to services or workers.

widgets/map_widget.py

Interactive survey map · 1,393 source lines

Responsibility

Projects geographic positions for display, draws tracklines and overlays, provides rectangle selection and point/interval picking, and reports cursor context.

Public shape

- Class `_SelectableViewBox`: `__init__`, `set_select_mode`, `mouseDragEvent`, `mouseClickEvent`
- Class `MapWidget`: `__init__`, `set_full_trackline`, `set_videos`, `set_pick_mode`, `load_data`, `set_segments`, `set_video_coverage_coloring`, `set_display_mode`, `set_select_mode`, `set_selection`, `clear_selection`, `fit_view`, `set_trackline_width`, `set_trackline_sensor_range`, `set_trackline_log_scale`, `set_nav_background_track`, `set_trackline_segment_ranges`, `set_anomaly_ranges`, `set_history_ranges`, `_update_reference` ...
- Module functions: `haversine_km`

Connections

Internal imports: `timeutil`

Important library imports: `PySide6.QtCore`, `PySide6.QtWidgets`, `datetime`, `math`, `numpy`, `pyqtgraph`, `typing`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
class _SelectableViewBox(pg.ViewBox):
    """A pyqtgraph ViewBox that adds rubber-band (drag) selection capability.

    When select_mode is active:
    - Left-button drag: emits selection_rect_changed(QRectF) continuously
      so the MapWidget can draw a live preview rectangle.
    - Left-button release: emits selection_committed(QRectF) with the final
      rectangle so MapWidget can compute which dots fall inside it.
    - Left-button click (no drag): emits selection_committed(None) to clear
      the selection.
    - Normal pan/zoom mouse events are suppressed during select_mode
      (setMouseEnabled(False)) to avoid accidental view changes while
      drawing a selection rectangle.

    When select_mode is False, all events are delegated to the normal
    ViewBox handler (pan, zoom, etc.).
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Keep this layer focused on presentation and translation between controls and dataclasses. Expensive reads and computations should move to services or workers.

widgets/navigation_import_dialog.py

Navigation mapping dialog · 602 source lines

Responsibility

Lets a user map arbitrary file columns to latitude, longitude, altitude, depth, heading, pitch, and roll sources.

Public shape

- Class `_SourceSection`: `__init__`, `_on_separate_toggled`, `_browse_file`, `prefill`, `_reload_without_headers`, `_populate_preview_table`, `_populate_column_combos`, `build_result`
- Class `NavigationImportDialog`: `__init__`, `_prefill_from`, `_build_ui`, `_on_accept`, `get_result`

Connections

Internal imports: `models`, `sensor_service`

Important library imports: `PySide6.QtCore`, `PySide6.QtWidgets`, `pathlib`, `re`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
class _SourceSection(QWidget):
    """A self-contained form for configuring one coordinate timeseries source.

    Used three times inside NavigationImportDialog – once for latitude, once
    for longitude, and once for altitude/depth. All three have the same
    layout; only the group box title and the "required" flag differ.

    UI elements:
    path_edit      – text field showing the selected file path
    browse_button  – opens a file chooser dialog
    timestamp_combo – dropdown for the timestamp column
    separate_check – checkbox: "Use separate date and time columns"
    date_combo     – secondary dropdown for the date column (hidden by default)
    value_combo    – dropdown for the lat/lon/alt value column
    preview_table  – read-only table showing the first 5 rows of the file
    headerless_warning – yellow warning label shown for headerless files
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Keep this layer focused on presentation and translation between controls and dataclasses. Expensive reads and computations should move to services or workers.

widgets/sensor_import_dialog.py

Sensor mapping dialog · 487 source lines

Responsibility

Previews a sensor file and maps its timestamp and value columns into one or more named SensorChannel objects.

Public shape

- Class SensorImportDialog: `__init__`, `_build_ui`, `_browse_file`, `_reload_without_headers`, `_on_separate_toggled`, `_update_timestamp_preview`, `_populate_preview_table`, `_populate_column_combos`, `_sync_default_name`, `_on_accept`, `get_result`

Connections

Internal imports: `models`, `sensor_service`

Important library imports: `PySide6.QtCore`, `PySide6.QtWidgets`, `pandas`, `pathlib`, `re`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
class SensorImportDialog(QDialog):
    """Modal dialog for importing one sensor data channel from a CSV or PPI file.

    Workflow:
    1. User browses to a CSV/PPI file. The first 5 rows are shown in a preview
       table, and all column headers are loaded into the combo boxes.
    2. User selects the timestamp column. A live parse preview immediately
       shows the interpreted UTC start and end times so timestamp mismatches
       are caught before running the pipeline.
    3. User selects the value column (e.g. "Temperature_C") and optionally
       provides a channel display name and units.
    4. User clicks OK → validation runs → SensorFileConfig is built and stored.
    5. Caller retrieves the result via get_result().

    The dialog adds exactly one SensorChannel per invocation. Channels from
    the same file can only be added by opening the dialog again.
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Keep this layer focused on presentation and translation between controls and dataclasses. Expensive reads and computations should move to services or workers.

widgets/timeline_widget.py

Temporal overview · 185 source lines

Responsibility

Draws video coverage, sensor availability, and selected time ranges on a linked pyqtgraph timeline.

Public shape

- Class TimelineWidget: `__init__`, `clear_timeline`, `set_data`, `_draw_segment`

Connections

Internal imports: `models`

Important library imports: `PySide6.QtCore`, `datetime`, `pyqtgraph`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
class TimelineWidget(PlotWidget):
    """Layered horizontal timeline showing video, navigation, and sensor coverage.

    Inherits from pyqtgraph's PlotWidget so it slots directly into a Qt layout
    without any extra wrapper. Each call to set_data() rebuilds the display
    from scratch: it clears all items, draws the coverage bars, and places
    row labels.

    Visual design:
    - Dark background (#1e1e1e) to contrast with the coloured bars.
    - Subtle grid lines (alpha=0.25) to aid time-reading without clutter.
    - Y axis tick marks hidden (showValues=False) because rows are labelled
      by TextItem rather than numeric tick values.

    Usage:
    timeline = TimelineWidget()
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Keep this layer focused on presentation and translation between controls and dataclasses. Expensive reads and computations should move to services or workers.

workspace_panel.py

Workspace browser · 1,061 source lines

Responsibility

Displays source files and versioned outputs, summarizes metadata, preserves tree expansion, and opens artifacts in native tools.

Public shape

- Class WorkspacePanel: `__init__`, `_build_ui`, `set_root`, `refresh`, `get_context_summary`, `set_sources`, `_populate_tree`, `_add_source_subtree`, `_add_outputs_subtree`, `_add_photogrammetry_subtree`, `_photo_run_summary`, `_add_versioned_runs`, `_run_size`, `_add_run_files`, `_add_legacy_products`, `update_status_strip`, `_on_tree_context_menu`, `_photo_run_context_menu`, `_open_path_with_os`, `_group_node` ...
- Module functions: `_fmt_size`, `_fmt_mtime`, `_read_meta`, `_settings_summary`

Connections

Internal imports: none. This is a leaf or external-facing module.

Important library imports: `PySide6.QtCore`, `PySide6.QtGui`, `PySide6.QtWidgets`, `datetime`, `json`, `os`, `pathlib`, `subprocess`, `sys`, `typing`

Qt boundary

This module belongs to the GUI/worker side. Widget state must be touched only on the main Qt thread.

Representative source

```
class WorkspacePanel(QWidget):
    """Read-only workspace file-structure viewer with per-run metadata.

    Signals:
        run_selected(str) — emitted when the user clicks a 3D model run node;
                           carries the absolute path of that run directory.
    """

    run_selected          = Signal(str) # user clicked a 3D model run node
    make_slices_requested = Signal(str) # user chose "Make Slices" from context menu
    open_in_viewer_requested = Signal(str) # "Open in 3D Viewer" — path to a PLY/OBJ
    open_in_metashape_requested = Signal(str) # "Open in Metashape GUI" — path to a .psx

    def __init__(self, parent=None):
        super().__init__(parent)
        self._root: Optional[str] = None
```

Excerpt copied from the checked-out source. Read surrounding code before changing behavior; the excerpt illustrates structure, not the entire contract.

Maintenance notes

Keep this layer focused on presentation and translation between controls and dataclasses. Expensive reads and computations should move to services or workers.

9. MATLAB/Python boundary

MATLAB is optional at application startup but required to calculate a fresh anomaly detector matrix. `anomaly_service` locates the executable and invokes `matlab -batch` in a controlled working directory. The exporter scripts write event CSVs; `build_anomaly_site_catalog.py` consumes those files. This disk contract keeps catalog generation testable and repeatable independently of MATLAB.

Why the boundary matters

Do not replace the event CSV seam with in-memory MATLAB objects unless there is a compelling reproducibility plan. Files provide provenance, debuggability, and graceful degradation.

ExportDetectorFamilyEvents.m

60 source lines

Exports detector-family event intervals into the CSV contract consumed by Python.

How it participates

This script defines the interchange format. Column names, timestamp formatting, channel names, and interval semantics must remain compatible with `build_anomaly_site_catalog.load_events`.

Representative source

```
if ~exist("repo","var") || strlength(string(repo)) == 0
    repo = "/home/troyboland/epr_imaging/epr_imaging";
end
repo = string(repo);
configs = ["masked","nomask_log","nomask_ceiling","nomask_both"];
matNames = [
    "grapher_matrix_results_masked.mat"
    "grapher_matrix_results_nomask_log.mat"
    "grapher_matrix_results_nomask_ceiling.mat"
    "grapher_matrix_results_nomask_both.mat"
];
methodNames = ["D1_height","D2_mass","D3_peak","D4_local"];
methodTokens = ["_D1_","_D2_","_D3_","_D4_"];
minFrac = 0.25;

for ci = 1:numel(configs)
    A = load(fullfile(repo,matNames(ci)),"results","runChannels","time_channel");
    outdir = fullfile(repo,"grapher_matrix_figs",configs(ci));
```

MATLAB excerpt from the checked-out script.

Beginner translation

MATLAB arrays are one-indexed, and table variables are accessed by names. A semicolon suppresses console output. Logical masks select matching rows. Functions such as `readtable` and `writetable` correspond roughly to `pandas.read_csv` and `DataFrame.to_csv` in Python.

ExportFineConsensusEvents.m

58 source lines

Exports fine-grained consensus event intervals for catalog fusion.

How it participates

This script defines the interchange format. Column names, timestamp formatting, channel names, and interval semantics must remain compatible with `build_anomaly_site_catalog.load_events`.

Representative source

```
if ~exist("repo","var") || strlen(string(repo)) == 0
    repo = "/home/troyboland/epr_imaging/epr_imaging";
end
repo = string(repo);
configs = ["masked","nomask_log","nomask_ceiling","nomask_both"];
matNames = [
    "grapher_matrix_results_masked.mat"
    "grapher_matrix_results_nomask_log.mat"
    "grapher_matrix_results_nomask_ceiling.mat"
    "grapher_matrix_results_nomask_both.mat"
];
minFrac = 0.50;

for ci = 1:numel(configs)
    A = load(fullfile(repo,matNames(ci)),"results","runChannels","time_channel");
    outdir = fullfile(repo,"grapher_matrix_figs",configs(ci));
    for ch = reshape(A.runChannels,1,[])
        R = A.results.(ch).fine;
```

MATLAB excerpt from the checked-out script.

Beginner translation

MATLAB arrays are one-indexed, and table variables are accessed by names. A semicolon suppresses console output. Logical masks select matching rows. Functions such as `readtable` and `writetable` correspond roughly to `pandas.read_csv` and `DataFrame.to_csv` in Python.

GrapherMatrix.m

1,622 source lines

Runs the strategy matrix across baselines, smoothers, detectors, channels, and masking regimes.

How it participates

This is the broad detector experiment driver. It combines baseline, smoothing, detector, and masking choices, calculates agreement/consensus, saves figures and MAT results, and calls the event exporters.

Representative source

```
filename = "interp_full.csv";

% Preprocessing (identical to Grapher.m so segments match).
speedThreshold = 0.05;
invalidBuffer = 30;
shortGapThreshold = 60;
navWindow = 300;
navSmooth = "gaussian";

% -----
% Two REGIMES of the same 5x4x4 engine.
%
% FINE = the original tuning (short transits): a 4000 s baseline window, tight
%       event limits. It MISSES large/long anomalies by construction - a plume
%       longer than ~half the baseline window is absorbed INTO the baseline (the
%       50% law / its frequency analogue), so the residual over it collapses and
%       the detector sees nothing. maxEventSec caps B2 at 1200 s outright.
%
```

MATLAB excerpt from the checked-out script.

Beginner translation

MATLAB arrays are one-indexed, and table variables are accessed by names. A semicolon suppresses console output. Logical masks select matching rows. Functions such as `readtable` and `writetable` correspond roughly to `pandas.read_csv` and `DataFrame.to_csv` in Python.

navSpeed.m

77 source lines

Derives navigation speed used to diagnose survey motion.

How it participates

This is an exploratory or supporting analysis used by the detector workflow. It operates on files in the analysis working directory rather than importing Python modules.

Representative source

```
function v = navSpeed(t, lat, lon, alt, opts)
%NAVSPEED  Vehicle speed (m/s) at each nav/channel row.
%
%   v = navSpeed(t, lat, lon, alt)
%
%   Nav and channel share a time axis, so this returns one speed per row,
%   aligned 1:1 with the input (same length, same order – nothing dropped).
%
% INPUTS
%   t       - timestamps (Unix seconds or datetime), N x 1
%   lat, lon - decimal degrees, N x 1
%   alt     - vertical position, metres, N x 1.  [] for horizontal-only.
%             Use DEPTH for vehicle motion; ALTITUDE tracks terrain and
%             adds vertical speed the vehicle didn't actually have.
%
% OPTIONS
%   Smooth - 'movmedian' (default) | 'gaussian' | 'none'
%   Window - smoothing window in samples (default 5)
```

MATLAB excerpt from the checked-out script.

Beginner translation

MATLAB arrays are one-indexed, and table variables are accessed by names. A semicolon suppresses console output. Logical masks select matching rows. Functions such as `readtable` and `writetable` correspond roughly to `pandas.read_csv` and `DataFrame.to_csv` in Python.

10. Installation, optional dependencies, and output

The normal developer entry point is `python app.py` after installing `requirements.txt`. The Windows installer automates Python and virtual-environment setup. Core imports include PySide6, pandas, NumPy, SciPy, OpenCV, rasterio, pykrige, netCDF4, Pillow, matplotlib, pyqtgraph, PyVista, and PyVistaQt. Metashape, COLMAP, MATLAB, and PotreeConverter are discovered at runtime and remain optional for unrelated workflows.

Workspace layout

<code>workspace/</code>	
<code>workspace.json</code>	saved model and settings
<code>interp_full.csv</code>	aligned whole-survey table
<code>inputs/ nav/ sensor/</code>	copied or referenced source material
<code>outputs/</code>	full-survey products
<code>job_001_name/</code>	
<code>filtered_interp.csv</code>	only rows in the Job's intervals
<code>filtered_interp.meta.json</code>	filtering provenance
<code>outputs/</code>	job-scoped products
<code>logs/task_log_TIMESTAMP.txt</code>	execution record

Safe extension checklist

- Add or change a dataclass field with a backward-compatible default and update JSON serialization.
- Keep scientific work in a Qt-free service and accept log/progress callables.
- Register a new product type in `TASK_INFO`, expose settings in `TaskConfigDialog`, expand it in `plan_service`, and dispatch it in `StackWorker`.
- Write output metadata containing source paths, settings, units, coordinate reference system, and software version.
- Connect worker signals to QObject methods, arrange quit/deleteLater cleanup, and test cancellation and error paths.
- Run byte compilation, static undefined-name checks, the Qt/threading linter, and pytest.

Known concentration of risk

`main_window.py` is a very large coordinator, so seemingly local UI edits can cross workspace, worker, and output concerns. `pipeline_service.py` and `photogrammetry_service.py` are long procedural workflows whose intermediate-file contracts matter. Scientific gridding and anomaly thresholds require domain validation in addition to ordinary code tests.

11. Testing and debugging

The current automated tests emphasize plan expansion and UTC semantics. Those are valuable because they protect two application-wide invariants without needing Qt or scientific libraries. A broader test strategy should add fixture-driven sensor parsing, small deterministic interpolation grids, workspace round trips, output metadata checks, and smoke tests for worker dispatch.

- Start a failure investigation with the task log and the relevant run's metadata JSON.
- Confirm the selected interp path and inspect its timestamp, latitude/longitude, UTM, depth/altitude, and requested sensor columns.
- Distinguish missing optional software from bad input data; detection helpers provide explicit availability reasons.
- For a map offset, verify coordinate system and UTM zone before tuning visualization.
- For an empty Job, compare interval UTC times against master-table coverage and video coverage.
- For a frozen or crashing GUI, audit worker signal connections and ensure no widget is touched from a worker callback.

A practical change trace

Suppose a new gridding option is added. Begin at the output algorithm and its metadata, then expose it in the service signature. Add a `Task.settings` key and dialog control. Confirm `plan_service` copies the key into the step. Confirm `StackWorker` forwards it. Finally, confirm workspace and preset serialization preserve it. This bottom-up trace follows the program's architecture and reveals missing links early.

Appendix A · Glossary

datetime

A Python object representing a date and clock time. In this program a datetime is deliberately timezone-naive but is interpreted as UTC.

DataFrame

A pandas table: named columns and numbered rows, similar to a spreadsheet but designed for code and large numerical datasets.

interpolation

Estimating a value between measured samples. It is what allows navigation, sensor readings, and video frames recorded at different instants to share one timeline.

UTM

A map projection that converts longitude/latitude angles to local easting and northing distances in metres.

raster

A rectangular grid of cells. Each cell stores a value such as depth, methane concentration, or temperature.

point cloud

A collection of points in 3-D space. Every point can also carry a scalar value or RGB color.

signal

Qt's event notification mechanism. A worker emits a signal; a slot on another object receives the data safely.

dataclass

A Python class optimized for records. Python generates its constructor and other routine methods from declared fields.

service

A module containing reusable application logic without the buttons and windows of the GUI.

GeoTIFF

A TIFF image whose metadata says where its pixels lie on Earth.

NetCDF

A portable scientific-data file containing named multidimensional arrays and metadata.

IDW

Inverse-distance weighting: nearby observations influence an estimated grid cell more than distant observations.

Job

A named collection of survey time intervals and a snapshot of processing settings.

Task

A user-editable declaration of a desired product, target, channels, and settings.

Scope

The planner's concrete interpretation of a target: input table, output directory, label, and optional Job.

plan step

A plain dictionary containing everything StackWorker needs for one unit of execution.

CLAHE

Contrast Limited Adaptive Histogram Equalization, an OpenCV method that improves local image contrast while limiting noise amplification.

photogrammetry

Recovering camera poses and 3-D structure from overlapping photographs.

nodata

A sentinel or mask indicating that a raster/grid cell has no valid observation or estimate.

Appendix B · Source inventory

- 3dvistool/example_usage.py — 94 lines; 0 classes; 1 module functions
- 3dvistool/point_cloud_pipeline.py — 515 lines; 1 classes; 1 module functions
- anomaly_service.py — 430 lines; 0 classes; 16 module functions
- app.py — 69 lines; 0 classes; 1 module functions
- build_anomaly_site_catalog.py — 1,911 lines; 0 classes; 39 module functions
- build_architecture_pdf.py — 479 lines; 0 classes; 20 module functions
- chat_panel.py — 424 lines; 1 classes; 1 module functions
- claude_service.py — 95 lines; 1 classes; 2 module functions
- comprehensive_software_documentation.py — 593 lines; 1 classes; 12 module functions
- config_service.py — 621 lines; 1 classes; 2 module functions
- dynamicsampling.py — 433 lines; 0 classes; 3 module functions
- frame_stats.py — 171 lines; 0 classes; 3 module functions
- interval_io.py — 171 lines; 0 classes; 4 module functions
- main_window.py — 8,845 lines; 6 classes; 2 module functions
- models.py — 804 lines; 14 classes; 0 module functions
- netcdf_export.py — 192 lines; 0 classes; 4 module functions
- one_click_dialog.py — 587 lines; 1 classes; 0 module functions
- output_service.py — 1,192 lines; 1 classes; 4 module functions
- photogrammetry_service.py — 1,467 lines; 0 classes; 31 module functions
- pipeline_service.py — 2,432 lines; 2 classes; 0 module functions
- plan_service.py — 473 lines; 3 classes; 8 module functions
- point_cloud_pipeline.py — 846 lines; 1 classes; 5 module functions
- preset_service.py — 138 lines; 0 classes; 11 module functions
- qc_report.py — 313 lines; 0 classes; 6 module functions
- qgis_export.py — 394 lines; 0 classes; 15 module functions
- sensor_service.py — 716 lines; 1 classes; 0 module functions
- stack_panel.py — 488 lines; 2 classes; 0 module functions
- stack_runner.py — 880 lines; 1 classes; 0 module functions
- task_config_dialog.py — 994 lines; 1 classes; 0 module functions
- timeutil.py — 49 lines; 0 classes; 2 module functions
- video_service.py — 315 lines; 2 classes; 0 module functions
- viewer_widget.py — 869 lines; 4 classes; 3 module functions
- widgets/annotation_settings_dialog.py — 302 lines; 1 classes; 0 module functions
- widgets/map_widget.py — 1,393 lines; 2 classes; 1 module functions
- widgets/navigation_import_dialog.py — 602 lines; 2 classes; 0 module functions
- widgets/sensor_import_dialog.py — 487 lines; 1 classes; 0 module functions
- widgets/timeline_widget.py — 185 lines; 1 classes; 0 module functions
- workspace_panel.py — 1,061 lines; 1 classes; 4 module functions
- ExportDetectorFamilyEvents.m — 60 lines; MATLAB analysis
- ExportFineConsensusEvents.m — 58 lines; MATLAB analysis
- GrapherMatrix.m — 1,622 lines; MATLAB analysis
- navSpeed.m — 77 lines; MATLAB analysis